

Міністерство освіти і науки України
Національний університет "Львівська політехніка"



Звіт до лабораторної роботи №9
З дисципліни "Web-технології та Web-дизайн"
на тему: "React.js: Connecting to REST API"

Виконала:

Ст.гр. ІР-11

Луцан І.З.

Прийняв:

доцент, к.т.н.

Іванюк О.О.

Завдання:

9. React.js: Connecting to REST API

Description: Finally! Now, you are about to put a final touches on all pages you created - implement interaction with your REST API server.

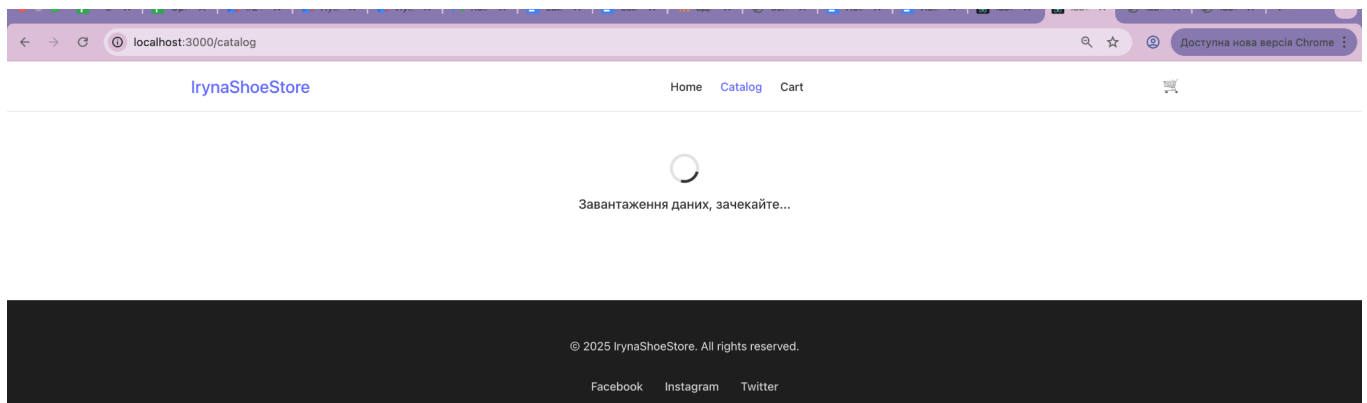
Variants - (products that you are 'selling') the same as for previous works. (see the description to 3rd work)

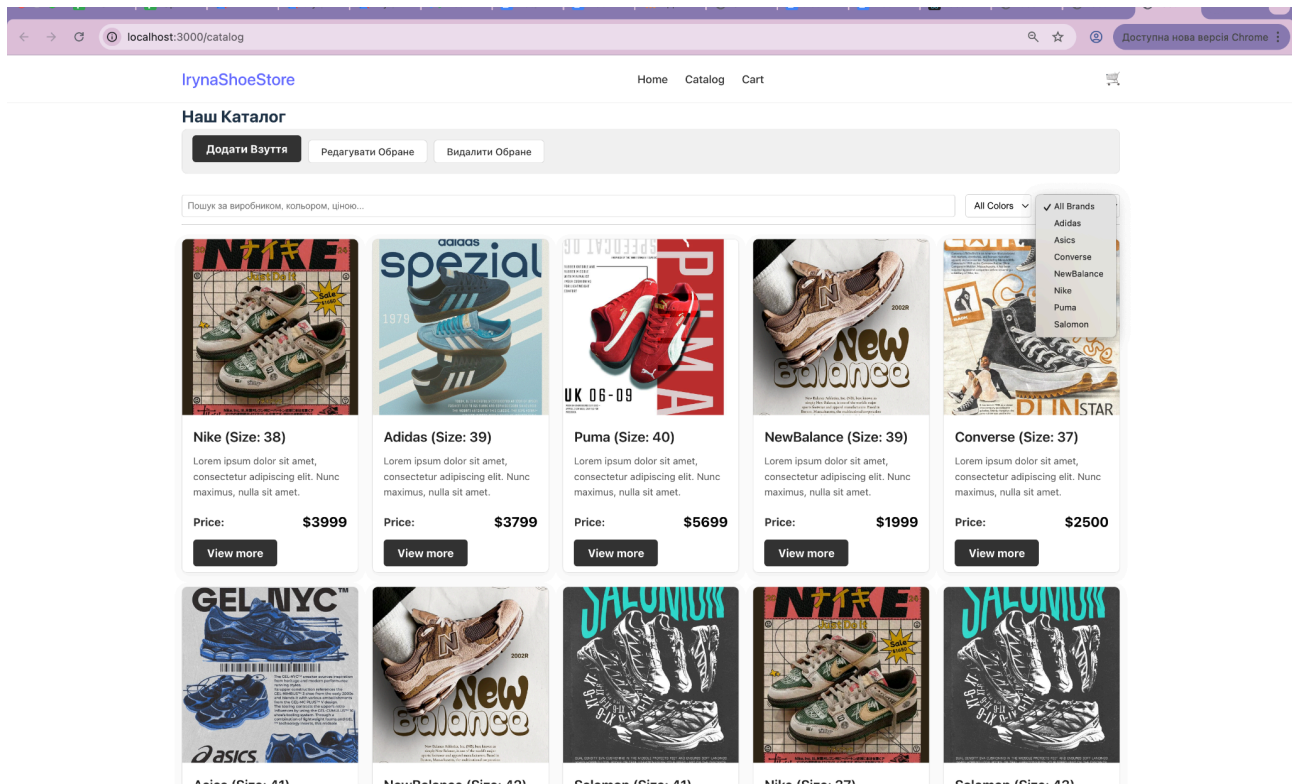
Backend - just as discussed before, can be the one you used for your **3-5 work** or a new one created **from scratch**. Tech stack - **absolutely up to you**.

Requirements:

- All of the requirements for previous React.js works should be kept.
- **Code style:**
 - For any **http request** - use **axios** library
<https://github.com/axios/axios#installing>
 - All your API functions should be separated into **single file (or folder, if you want)** - just like you saw in Live coding for 5 lab with **fetch()** function
- **Functionality:**
 - On **Catalog Page** - all items should now be fetched from your backend with GET method (using axios)
 - **Search with filters** - should also be implemented with GET request (search by text field can be left as it is)
*Hint: pass filters as **url parameter***
 - Before response from your GET method is received you have to display a **Spinner(Loader component)** to the user.
Something like this: <https://projects.lukehaas.me/css-loaders/>

Нові зміни на сайті:





Файл ShoesApi.js:

```
import axios from 'axios';

const BASE_URL = '/api/shoes';
const NEWS_URL = '/api/featured-news';

/**
 * 1. Отримати список взуття (з підтримкою фільтрів).
 * @param {object} [filters={}] - Об'єкт фільтрів (producer, color)
 * @returns {Promise<Array>}
 */
export async function fetchShoes(filters = {}) {
  // Axios автоматично перетворює об'єкт params на рядок запиту (?producer=Nike&color=black)
  const response = await axios.get(BASE_URL, {
    params: filters,
  });
  return response.data;
}

/**
 * 2. Отримати один товар за ID.
 * @param {number} id - ID товару.
 * @returns {Promise<object>}
 */
export async function fetchShoeById(id) {
```

```

    const response = await axios.get(`${BASE_URL}/${id}`);
    return response.data;
}

/**
 * 3. Створити новий товар (POST).
 * @param {object} shoeData - Дані нового товару.
 * @returns {Promise<object>}
 */
export async function createShoe(shoeData) {
    const response = await axios.post(BASE_URL, shoeData);
    return response.data;
}

/**
 * 4. Оновити існуючий товар (PUT).
 * @param {number} id - ID товару.
 * @param {object} shoeData - Дані для оновлення.
 * @returns {Promise<object>}
 */
export async function updateShoe(id, shoeData) {
    const response = await axios.put(`${BASE_URL}/${id}`, shoeData);
    return response.data;
}

/**
 * 5. Видалити товар за ID (DELETE).
 * @param {number} id - ID товару.
 * @returns {Promise<object>}
 */
export async function deleteShoe(id) {
    const response = await axios.delete(`${BASE_URL}/${id}`);
    return response.data;
}

// --- Функції для Новин (FeaturedNews) ---

/**
 * 6. Отримати список новин (для HomePage).
 * @returns {Promise<Array>}
 */
export async function fetchFeaturedNews() {
    const response = await axios.get(NEWS_URL);
    return response.data;
}

```

Файл Loader.jsx:

```
import './Loader.css';

function Loader() {
  return (
    <div className="loader-container">
      <div className="spinner"></div>
      <p>Завантаження даних, зачекайте...</p>
    </div>
  );
}

export default Loader;
```

Файл CatalogPage.jsx:

```
import React, { useState, useEffect } from 'react';
import ShoeCard from '../containers/Components/ShoeCard/ShoeCard';
import SecondaryButton from '../containers/Components/ui/SecondaryButton';
import PrimaryButton from '../containers/Components/ui/PrimaryButton';
import { Link, useNavigate } from 'react-router-dom';
import { fetchShoes, deleteShoe } from '../api/shoesApi';
import Loader from '../containers/Components/Loader/Loader.jsx';

const gridStyles = {
  display: 'grid',
  gridTemplateColumns: 'repeat(auto-fill, minmax(250px, 1fr))',
  gap: '20px',
  padding: '20px 0'
};

const filterBarStyles = {
  display: 'flex',
  flexWrap: 'wrap',
  gap: '15px',
  alignItems: 'center',
  padding: '10px 0'
};

const adminPanelStyles = {
  padding: '15px',
  backgroundColor: '#f4f4f4',
  border: '1px solid #ddd',
  borderRadius: '8px',
  display: 'flex',
  gap: '10px',
  marginBottom: '20px'
};

function CatalogPage() {
```

```

const navigate = useNavigate();
const [products, setProducts] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
const [selectedId, setSelectedId] = useState(null);
const [isEditDisabled, setIsEditDisabled] = useState(true);
const [isDeleteDisabled, setIsDeleteDisabled] = useState(true);
const [searchTerm, setSearchTerm] = useState('');
const [selectedColor, setSelectedColor] = useState('All');
const [selectedProducer, setSelectedProducer] = useState('All');

useEffect(() => {
  setIsEditDisabled(selectedId === null);
  setIsDeleteDisabled(selectedId === null);
}, [selectedId]);

// Обробник зміни фільтра кольору
const handleColorChange = (e) => {
  setSelectedColor(e.target.value);
};

const handleProducerChange = (e) => {
  setSelectedProducer(e.target.value);
};

const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));

// GET з фільтрами
const fetchProducts = async () => {
  try {
    setLoading(true);
    setError(null);

    // Створення об'єкта фільтрів для передачі як URL-параметрів (GET)
    const filters = {
      producer: selectedProducer !== 'All' ? selectedProducer : undefined,
      color: selectedColor !== 'All' ? selectedColor : undefined,
    };

    // API-список fetchShoes(filters)
    const data = await fetchShoes(filters);
    await delay(700);
    setProducts(data);
  } catch (err) {
    setError(err.message);
    console.error("Помилка завантаження взуття:", err);
  } finally {
    setLoading(false);
  }
};

// викликає fetchProducts при зміні фільтрів

```

```

useEffect(() => {
  fetchProducts();
}, [selectedProducer, selectedColor]);

const handleDelete = async () => {
  if (!selectedId) {
    alert("Будь ласка, оберіть товар для видалення.");
    return;
  }

  if (window.confirm('Ви впевнені, що хочете видалити цей товар?')) {
    try {
      // API-сервіс deleteShoe(id)
      await deleteShoe(selectedId);

      setSelectedId(null);
      alert('Товар успішно видалено!');
      fetchProducts(); // Перезавантаження списку
    } catch (err) {
      console.error("Помилка видалення:", err);
      alert("Не вдалося видалити товар: " + err.message);
    }
  }
};

const filteredProducts = products.filter(product => {

  const query = searchTerm.toLowerCase().trim();
  if (query === '') return true;

  // текстовий пошук
  const producerMatch = product.producer.toLowerCase().includes(query);
  const colorMatch = product.color.toLowerCase().includes(query);
  const priceMatch = String(product.price).includes(query);
  const sizeMatch = String(product.size).includes(query);

  return producerMatch || colorMatch || priceMatch || sizeMatch;
});

if (loading) return <Loader />;
if (error) return <p>Сталася помилка: {error}</p>;

return (
  <div>
    <h2>Наш Каталог</h2>

    <div style={adminPanelStyles}>
      <Link to="/catalog/new" style={{ textDecoration: 'none' }}>

```

```

        <PrimaryButton as="span">
            Додати Взуття
        </PrimaryButton>
    </Link>

    <Link
        to={` /catalog/edit/${selectedId}`}
        style={{ textDecoration: 'none' }}
    >
        <SecondaryButton as="span" disabled={isEditDisabled}>
            Редагувати Обране
        </SecondaryButton>
    </Link>

    <SecondaryButton
        disabled={isDeleteDisabled}
        style={!isDeleteDisabled ? {backgroundColor: '#ffdddd', borderColor: '#ff5555', color:
'#d00'} : {}}
        onClick={handleDelete}
    >
        Видалити Обране
    </SecondaryButton>
</div>

<div style={filterBarStyles}>
    <input
        type="text"
        placeholder="Пошук за виробником, кольором, ціною..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        style={{ padding: '8px', border: '1px solid #ccc', borderRadius: '4px', flexGrow: 1 }}
    />

    <select
        value={selectedColor}
        onChange={handleColorChange}
        style={{ padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }}
    >
        <option value="All">All Colors</option>
        {products.map(p => p.color).filter((v, i, a) => a.indexOf(v) === i).sort().map(color
=> (
            <option key={color} value={color}>{color}</option>
        ))}
    </select>

    <select
        value={selectedProducer}
        onChange={handleProducerChange}
        style={{ padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }}
    >

```



```

      <option value="All">All Brands</option>
      {products.map(p => p.producer).filter((v, i, a) => a.indexOf(v) ===
i).sort().map(producer => (
        <option key={producer} value={producer}>{producer}</option>
      ))}
    </select>

</div>
<hr />

<div style={gridStyles}>
  {filteredProducts.map(product => (
    <ShoeCard
      key={product.id}
      product={product}
      onSelect={() => {
        setSelectedId(prevId => prevId === product.id ? null : product.id);
      }}
      isSelected={product.id === selectedId}
    />
  ))}
  {filteredProducts.length === 0 && <p>За вашим запитом нічого не знайдено.</p>}
</div>
</div>
);
}

export default CatalogPage;

```

Файл ProductDetailPage:

```

import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { fetchShoeById } from '../api/shoesApi';
import Loader from '../containers/Components/Loader/Loader.jsx';

import PrimaryButton from '../containers/Components/ui/PrimaryButton';
import SecondaryButton from '../containers/Components/ui/SecondaryButton';

import Adidas from '../assets/Adidas.jpg';
import Asics from '../assets/Asics.jpg';
import Converse from '../assets/Converse.jpg';
import Newbalance from '../assets/Newbalance.jpg';
import Nike from '../assets/Nike.jpg';
import Puma from '../assets/Puma.jpg';
import Salomon from '../assets/Salomon.jpg';
import DefaultImage from '../assets/shoes_store.jpg';

const sharedStyles = {

```

```

mainContainer: {
  display: 'flex',
  gap: '40px',
  padding: '40px 0',
  maxWidth: '1200px',
  margin: '0 auto',
  backgroundColor: '#fff',
},
imageContainer: {
  flex: '1',
  minWidth: '400px',
  boxShadow: '0 4px 8px rgba(0,0,0,0.1)',
  borderRadius: '10px',
  overflow: 'hidden',
},
image: {
  width: '100%',
  height: 'auto',
  objectFit: 'cover',
},
details: {
  flex: '1',
  display: 'flex',
  flexDirection: 'column',
  justifyContent: 'center',
},
action: {
  marginTop: '30px',
  display: 'flex',
  gap: '15px',
  alignItems: 'center',
},
price: {
  fontSize: '2.5em',
  fontWeight: 'bold',
  color: '#333',
  margin: '10px 0',
}
};

```

```

function ProductDetailPage() {
  const { id } = useParams();
  const navigate = useNavigate();

  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {

```

```

const fetchProduct = async () => {
  try {
    setLoading(true);
    setError(null);

    const data = await fetchShoeById(id);
    setProduct(data);

  } catch (err) {
    if (err.response && err.response.status === 404) {
      setError('Товар не знайдено.');
```

} else {

```
      setError(`Помилка завантаження даних: ${err.message}`);
      console.error(`Помилка завантаження товару ${id}:`, err);
    }
  } finally {
    setLoading(false);
  }
};

fetchProduct();
}, [id]);

// Логіка вибору зображення
const getImageUrl = (producerName) => {
  switch (producerName?.toLowerCase()) {
    case 'nike': return Nike;
    case 'adidas': return Adidas;
    case 'puma': return Puma;
    case 'new balance': return Newbalance;
    case 'converse': return Converse;
    case 'asics': return Asics;
    case 'salomon': return Salomon;
    default: return DefaultImage;
  }
};

// УМОВНИЙ РЕНДЕРИНГ
if (loading) return <Loader />;
if (error) return <p style={{ padding: '20px', textAlign: 'center', color: 'red' }}>Сталася помилка: {error}</p>;
if (!product) return <p style={{ padding: '20px', textAlign: 'center' }}>Товар не знайдено.</p>;

// Якщо товар успішно завантажено, відображаємо його
return (
  <div style={sharedStyles.mainContainer}>
    <div style={sharedStyles.imageContainer}>
      <img src={getImageUrl(product.producer)} alt={product.producer}
style={sharedStyles.image} />
    </div>
  </div>
);

```

```

</div>
<div style={sharedStyles.details}>
  <h1>{product.producer} {product.color} Sneaker</h1>
  <p style={{ fontSize: '1.2em', color: '#666' }}>
    **Size:** {product.size} | **Color:** {product.color}
  </p>
  <div style={sharedStyles.price}>${product.price.toFixed(2)}</div>

  <p>
    Це детальний опис товару, який може бути розширений.
    Наразі він імітує реальний контент для демонстрації.
    Це взуття ідеально підходить для **{product.producer}** ентузіастів.
  </p>

  <div style={sharedStyles.action}>
    <PrimaryButton>
      Купити зараз
    </PrimaryButton>
    <SecondaryButton onClick={() => navigate('/catalog')}>
      Повернутися до каталогу
    </SecondaryButton>
  </div>
</div>
</div>
);
}

export default ProductDetailPage;

```

Посилання на лабораторну роботу на гітхабі:

https://github.com/IrynaLutsan/Web_Programming_Labs/tree/main/Labs_React

Висновок:

Під час виконання лабораторної роботи я реалізувала взаємодію клієнтської частини React-додатку із REST API сервером, використовуючи бібліотеку axios для надсилання асинхронних запитів. Відповідно до завдання, усі API-функції для отримання, створення, оновлення та видалення даних були винесені в окремий файл, що дозволило структурувати логіку спілкування з бекендом. Я налаштувала відображення товарів у каталозі через GET-запит із підтримкою фільтрації через URL-параметри, а також реалізувала перегляд окремого товару за його ID. Для покращення користувацького інтерфейсу було додано компонент Loader, який показує стан завантаження до отримання відповіді від сервера, та забезпечено обробку помилок у разі збоїв з'єднання. Також було успішно імплементовано функцію видалення товарів, що автоматично оновлює список на сторінці без необхідності її перезавантаження.